DAMG 7245

Big-Data Systems and Intelligence Analytics

Final Project Proposal

Industrial Digital Twin: Big Data-Driven Factory Simulation & LLM-Powered Predictive Analytics

Team Members

Ayush Patil	33.3%
Piyush Kunjilwar	33.3%
Raghavendra Prasath Sridhar	33.3%

Attestation

WE ATTEST THAT WE HAVEN'T USED ANY OTHER STUDENTS' WORK IN OUR ASSIGNMENT AND ABIDE BY THE POLICIES LISTED IN THE STUDENT HANDBOOK.

Northeastern University — Spring 2026

Contents

1 Title **4**

2 Introduction **4**
2.1 Background . . . 4
2.2 Objective . . . 5

3 Project Overview **5**
3.1 Scope . . . 5
3.1.1 In-Scope . . . 5
3.1.2 Out-of-Scope . . . 6
3.2 Stakeholders / End Users . . . 7

4 Problem Statement **7**
4.1 Current Challenges . . . 7
4.2 Opportunities . . . 8

5 Methodology **8**
5.1 Data Sources . . . 8
5.2 Technology Stack . . . 10
5.3 Architecture . . . 13
5.3.1 Data Flow Diagram . . . 14
5.4 Data Processing & Transformation . . . 15
5.4.1 Batch Processing (Spark on EMR) . . . 15
5.4.2 Streaming Processing . . . 15
5.4.3 Embedding Generation Pipeline . . . 15
5.5 LLM Integration Strategy . . . 16
5.5.1 Multi-Provider LLM Routing (LiteLLM) . . . 16
5.5.2 Agentic Architecture (LangGraph) . . . 16
5.5.3 Tool Definitions (LangChain) . . . 17
5.5.4 MCP Server Integration . . . 18
5.5.5 Use Case Summary . . . 18
5.6 Guardrails & Human-in-the-Loop (HITL) . . . 19
5.7 Evaluations & Testing . . . 19
5.7.1 LLM Evaluation Framework . . . 19
5.7.2 Software Testing . . . 19
5.7.3 Key Metrics . . . 20
5.8 Proof of Concept (POC) . . . 20

6 Project Plan & Timeline **21**
6.1 Milestones . . . 21
6.2 Timeline . . . 21

7 Team Roles & Responsibilities **22**

8 Risks & Mitigation **22**

8.1 Risk Assessment Matrix . . . 23

9 Expected Outcomes & Metrics **23**
9.1 KPIs . . . 23
9.2 Expected Benefits . . . 23

10 Token & Cost Report **24**
10.1 Token Consumption Tracking . . . 24
10.2 Cost Drivers . . . 24
10.3 Optimization Strategy . . . 24
10.4 Estimated Budget . . . 25

11 Conclusion **25**

12 References **26**

Appendix **27**
A. System Architecture Diagram (Mermaid) . . . 27
B. Pydantic Response Schema . . . 28
C. Sample Airflow DAG Structure . . . 28

1 Title

Industrial Digital Twin: Big Data-Driven Factory Simulation & LLM-Powered Predictive Analytics Platform

A cloud-native platform that ingests multi-source industrial IoT sensor data at scale, processes it through distributed Spark pipelines, and leverages LLM-powered Retrieval-Augmented Generation (RAG) to transform raw telemetry into actionable manufacturing intelligence—all accessible through an interactive Digital Twin user interface.

2 Introduction

2.1 Background

Modern manufacturing generates enormous volumes of data from IoT sensors, programmable logic controllers (PLCs), SCADA systems, maintenance logs, and quality inspection records. The McKinsey Global Institute estimates that the average smart factory produces over 1 petabyte of data annually, yet less than 1% of this data is actively analyzed for operational decision-making. This creates a massive information asymmetry: the data to prevent failures, optimize throughput, and reduce costs *already exists*, but it remains trapped in siloed systems, incompatible formats, and legacy databases.

The consequences are significant. Unplanned manufacturing downtime costs U.S. industry an estimated $50 billion annually. Reactive maintenance—waiting for equipment to fail before acting—accounts for over 60% of maintenance activity in small and mid-size manufacturers (SMBs). Root cause analysis after a failure event typically takes 4–48 hours of expert engineer time, during which production may be halted or degraded.

Digital Twin technology—creating virtual replicas of physical factory systems calibrated with real sensor data—has emerged as a transformative approach. Companies like Siemens (MindSphere), GE (Predix), and PTC (ThingWorx) offer enterprise Digital Twin platforms. However, these solutions present critical barriers for SMBs:

- **Cost:** Enterprise licenses range from $100K–$1M+ annually
- **Complexity:** Require dedicated IoT infrastructure teams and months of integration
- **Intelligence Gap:** Existing platforms provide visualization and basic alerting but lack integration with modern Large Language Models for contextual interpretation of anomalies
- **Vendor Lock-in:** Proprietary data formats and closed ecosystems

Simultaneously, the emergence of powerful LLMs with large context windows and strong reasoning capabilities opens a new paradigm: *natural language manufacturing intelligence*. Instead of engineers manually correlating sensor readings with equipment manuals, an LLM can ingest both data streams and produce contextual explanations, predictions, and recommendations—in seconds rather than hours.

This project bridges both gaps by building an **open, cloud-native Digital Twin platform** powered by big data engineering pipelines and LLM-driven analytics, accessible to manufacturers of any size.

2.2 Objective

The primary objectives of this project are:

1. **Big Data Engineering:** Build a scalable pipeline (Spark + Airflow) to ingest, clean, and transform 50+ GB of multi-source industrial IoT sensor data, with feature engineering for predictive analytics
2. **Significant LLM Integration:** Implement four core LLM workflows using Claude API—RAG-based anomaly explanation, natural language factory queries (NL-to-SQL), what-if scenario analysis, and automated maintenance report generation
3. **Cloud-Native Architecture:** Deploy across AWS (S3 data lake, EMR for Spark) and GCP (Cloud Run for application hosting, BigQuery for analytics) with Airflow orchestration
4. **User-Facing Application:** Deliver a React-based Digital Twin interface with interactive factory floor design, time-based simulation, multi-scenario comparison dashboards, and an AI-powered manufacturing advisor

Key Differentiator

Unlike existing enterprise Digital Twin platforms that cost $100K+ and offer no LLM integration, our platform delivers natural language manufacturing intelligence at a fraction of the cost (~$50/month infrastructure), making advanced factory analytics accessible to small and mid-size manufacturers.

3 Project Overview

3.1 Scope

3.1.1 In-Scope

- **Multi-source data acquisition:** Seven real-world data sources totaling 45+ GB—SEC EDGAR 10-K filings from 50+ manufacturers (~8 GB, parsed via pdfplumber), OSHA inspection database (4M+ records, ~12 GB via REST API), DOE Industrial Assessment Center audits (20K+ real facility assessments, ~2 GB), large-scale equipment catalog scraping from Grainger/Fanuc/ABB (~15 GB via async httpx), FRED + EIA economic and energy APIs (~500 MB time-series), EPA Toxics Release Inventory (25K+ facilities, ~5 GB), and engineering forum knowledge base scraped from Stack Exchange/Reddit (~3 GB)
- **Apache Spark ETL pipelines** on AWS EMR for batch processing—schema normalization, time-series feature engineering (rolling aggregates, FFT, statistical summaries), anomaly scoring (Isolation Forest, CUSUM, Z-score), and Parquet partitioning
- **Apache Airflow DAGs** for pipeline orchestration, scheduling, monitoring, and dependency management

- **Streaming pipeline:** Simulated IoT generator → Kafka → Spark Structured Streaming → S3/BigQuery for near-real-time anomaly detection
- **Vector embedding pipeline:** Equipment manual parsing (PyMuPDF + Unstructured) → recursive chunking (512 tokens, 64 overlap) → Voyage AI embeddings → Pinecone indexing with metadata filters
- **LLM integration via Claude API:** RAG anomaly explanation, NL-to-SQL factory queries, scenario optimization analysis, automated weekly maintenance reports
- **Guardrails & HITL:** Pydantic schema enforcement, hallucination detection, confidence scoring, prompt injection screening, human approval workflows for high-impact recommendations
- **FastAPI backend** on GCP Cloud Run with RESTful endpoints, WebSocket simulation streaming, rate limiting, and authentication
- **React Digital Twin frontend** with four core views—Factory Floor, Simulation, Scenarios, and AI Advisor
- **CI/CD** via GitHub Actions with automated unit, integration, and load tests
- **Observability:** Prometheus + Grafana for pipeline health, LLM latency, token usage, and cost dashboards

3.1.2 Out-of-Scope

- Physical IoT sensor hardware integration (simulated via Python generators)
- 3D rendering, CAD model visualization, or AR/VR interfaces
- PLC/SCADA protocol integration (OPC-UA, Modbus, MQTT from physical devices)
- Multi-tenant SaaS deployment with user authentication/authorization beyond basic API keys
- Training custom ML models from scratch (we leverage pre-trained embeddings and LLM reasoning)

3.2 Stakeholders / End Users

Stakeholder	Role	Benefit
Plant Managers	Primary user	Real-time factory performance dashboards; what-if simulation before capital expenditure; data-driven shift scheduling
Maintenance Engineers	Power user	LLM-powered anomaly explanations with equipment manual references; predictive maintenance alerts; natural language troubleshooting
Operations Analysts	Data consumer	Cost optimization insights; production line comparison; natural language queries over factory data ("Which line had the highest defect rate last Tuesday?")
Supply Chain Teams	Secondary user	Throughput forecasting for procurement planning; bottleneck identification for supplier negotiations
C-Suite Executives	Executive sponsor	High-level KPI dashboards; ROI analysis of machine investment scenarios; weekly automated intelligence reports

Table 1: Stakeholder analysis and value proposition

4 Problem Statement

4.1 Current Challenges

1. **Data Fragmentation:** Sensor data from different machine types resides in siloed systems—historians, SCADA databases, CSV exports, spreadsheets—with incompatible schemas, sampling rates, and timestamp formats. A single factory may have 15+ data formats across its production lines, with no unified query interface.

2. **Manual Diagnostic Workflows:** When a CNC mill's vibration sensor spikes, the current workflow is: (1) engineer reviews the alert, (2) manually cross-references the machine's maintenance manual, (3) checks historical logs for similar patterns, (4) consults with colleagues who have tribal knowledge. This process takes 4–48 hours—during which the machine may be offline.

3. **Lack of Contextual Intelligence:** Existing SCADA dashboards show raw metrics (temperature: 87°C, vibration: 2.4 mm/s) but provide zero interpretation. The dashboard shows a red number; it doesn't explain *why* the reading is abnormal or *what to do next*. This requires a manufacturing engineer's expertise that is increasingly scarce.

4. **Big Data Processing Bottlenecks:** A single production line with 50 sensors sampling at 1 Hz generates 4.3 million data points per day. Across a factory with 10 lines, that's 43M points/day or ~15 GB/month of raw sensor data. Traditional RDBMS and Excel-based analysis cannot scale to this volume for feature engineering and pattern detection.

5. **Simulation Gap:** Testing production changes in the physical world (adding machines, redistributing workers, changing shift schedules) is expensive, time-consuming, and disruptive. There is no accessible, data-driven simulation tool that lets managers preview the impact of decisions before committing real resources.

6. **Knowledge Attrition:** As experienced maintenance engineers retire, their institutional knowledge of equipment behavior, failure modes, and repair procedures leaves with them. This "tribal knowledge" is not captured in any searchable, queryable system.

4.2 Opportunities

Opportunity	Solution	Impact
Scalable processing	Apache Spark on AWS EMR	Process 50+ GB sensor data in <15 min
Contextual intelligence	RAG over manuals via Claude API	Root cause explanation in seconds, not hours
Unified data access	BigQuery analytical warehouse	Single SQL interface across all sensor sources
Natural language queries	LLM-powered NL-to-SQL	Non-technical users query factory data in English
Decision simulation	Data-calibrated Digital Twin engine	Test scenarios in 30 seconds vs. days physically
Knowledge preservation	Embedded manuals in vector store	Institutional knowledge becomes searchable AI

Table 2: Opportunities and their projected impact

5 Methodology

5.1 Data Sources

All data sources require non-trivial acquisition pipelines—API pagination, async web scraping, PDF parsing, deduplication, and cross-source schema normalization. There are no pre-cleaned CSV downloads.

Source	Acquisition	Size	Description & Engineering Challenge
SEC EDGAR 10-K/10-Q Filings	sec-edgar-downloader + pdfplumber + httpx	~8 GB	Annual/quarterly reports from 50+ manufacturers (Boeing, Caterpillar, 3M, Honeywell, Emerson, Parker Hannifin). Parse unstructured financial text to extract capex, equipment depreciation, maintenance expenses, production volumes. Requires NLP entity extraction with regex and keyword classification from 200+ page PDF filings.
OSHA Inspection & Accident Database	REST API with pagination + SHA-256 dedup	~12 GB	4M+ workplace inspection records with equipment-specific violations, injury details, SIC/NAICS manufacturing codes, citations, and penalty amounts. Bulk paginated API ingestion; SHA-256 content hashing for deduplication across incremental loads. Cross-reference with establishment data for facility-level failure pattern analysis.
DOE Industrial Assessment Center (IAC)	Bulk download + complex schema joins	~2 GB	20,000+ real energy audits of manufacturing facilities by DOE engineers. Actual equipment types, energy consumption, recommended improvements, implementation costs, and measured savings. The richest public source of real factory operational data—calibrates the simulation engine with ground-truth cost and efficiency parameters.
Equipment Manufacturer Catalogs	httpx async crawler + BeautifulSoup	~15 GB	Large-scale scraping of industrial catalogs (Grainger 1.5M+ products, AutomationDirect, Fanuc, ABB, Siemens Industrial). Extract machine specifications, pricing, power consumption, maintenance intervals, spare parts lists, and technical documentation PDFs. Async crawling with rate limiting and robots.txt compliance.
FRED + EIA Economic & Energy APIs	REST APIs + Airflow daily refresh	~500 MB	Federal Reserve: manufacturing PMI, industrial production index, commodity prices (steel, aluminum, copper), labor cost indices. EIA: real-time electricity prices by state, natural gas rates, industrial energy consumption. Feeds the Digital Twin simulation with live market data instead of hardcoded cost constants.
EPA Toxics Release Inventory (TRI)	Bulk CSV + API + geospatial enrichment	~5 GB	Facility-level environmental compliance data for 25,000+ manufacturing sites. Chemical usage, emissions, waste management, and compliance costs. Cross-joined with OSHA data by facility ID to build comprehensive plant profiles for environmental cost modeling.
Engineering Forums Knowledge Base	httpx + BeautifulSoup + NLP classification	~3 GB	Scrape public engineering Q&A (Stack Exchange Engineering, Reddit r/PLC, r/IndustrialMaintenance) for equipment troubleshooting threads and failure pattern discussions. Keyword-based classification filters relevant content. Chunked and embedded alongside manufacturer manuals for the RAG knowledge base.

Table 3: Sophisticated data sources requiring real data engineering pipelines

Total Data Volume & Complexity

45+ GB across 7 real-world sources. Every source requires a distinct acquisition strategy: API pagination with rate limiting (OSHA, FRED, EIA), large-scale async web scraping (equipment catalogs, forums), structured financial document parsing (SEC EDGAR), and bulk government data processing (DOE IAC, EPA TRI). Cross-source joins by facility ID, NAICS code, and equipment type create a unified manufacturing intelligence dataset that no single source provides alone. This is a genuine big-data integration challenge requiring Spark for distributed processing and Airflow for pipeline orchestration.

5.2 Technology Stack

Table 4: Complete technology stack—incorporating all DAMG 7245 coursework tools and frameworks

Layer	Technology	Justification
Data Ingestion & Scraping		
Web Scraping	httpx, BeautifulSoup	Async HTTP client (httpx) for non-blocking scraping of equipment manuals; BeautifulSoup for HTML parsing and section extraction
Doc Parsing	pdfplumber, regex	pdfplumber for structured PDF text/table extraction from maintenance manuals; regex for section boundary detection
Deduplication	hashlib (SHA-256)	Content-hash deduplication prevents re-processing unchanged manual versions
Cloud Storage	AWS S3	Data lake for raw sensor data (landing zone) and processed Parquet files (curated zone)
Orchestration	Apache Airflow	DAG-based pipeline scheduling with PythonOperator, XCom for inter-task data, cron triggers, exponential backoff retries
Streaming	Apache Kafka	IoT sensor message bus; Spark Structured Streaming consumer for near-real-time anomaly detection
Processing & Storage		
Batch Compute	Apache Spark (PySpark) on AWS EMR	Distributed ETL of 50+ GB sensor data; window functions for time-series features; MLlib for Isolation Forest anomaly scoring
Analytical DB	Snowflake	Analytical warehouse with VARIANT type for semi-structured sensor JSON, compound constraints, and Python Connector for pipeline integration
Caching	Redis	TTL-based response caching (cache-aside pattern), LLM output caching, cache invalidation via scan_iter, session state management
Precision	Python Decimal module	ROUND_HALF_UP quantization for cost calculations, weighted mean/std dev, score normalization

Continued on next page

Layer	Technology	Justification
Data Wrangling	Pandas DataFrames	EDA, feature inspection, .style formatting for Streamlit admin dashboard tables
Embedding & Retrieval		
Local Embeddings	Sentence Transformers (all-MiniLM-L6-v2)	384-dim vectors for development; zero-cost local embedding generation with fast iteration
Vector Store	Pinecone (managed, cosine, metadata filtering)	Managed vector database with low-latency similarity search, metadata filtering for equipment_type, auto-scaling, and serverless deployment
Sparse Search	rank_bm25 (BM25Okapi)	Lexical keyword search over technical terms that dense embeddings miss (part numbers, error codes)
Hybrid Retrieval	Dense + Sparse + RRF	Reciprocal Rank Fusion merges vector and BM25 results; best-of-both-worlds retrieval for technical documents
Query Expansion	HyDE	Hypothetical Document Embeddings: LLM generates a hypothetical answer, embeds it, then retrieves—improves recall for complex diagnostic queries
Chunking	Semantic/overlapping window	Configurable chunk_size (512), chunk_overlap (64), min_chunk_size; preserves cross-paragraph context in equipment manuals
Intelligence Layer (LLM + Agents)		
LLM Routing	LiteLLM	Multi-provider routing with automatic fallbacks (Claude Sonnet → OpenAI GPT-4o → Google Gemini), per-model cost tracking, budget management, temperature/max_tokens config per task
LLM Providers	Claude Sonnet + OpenAI GPT-4o + Gemini + Claude Haiku	Sonnet (primary) for RAG analysis; GPT-4o for NL-to-SQL; Gemini as secondary fallback; Haiku for lightweight classification. Via LangChain ChatAnthropic, ChatOpenAI, and ChatGoogleGenerativeAI
Agentic Framework	LangGraph	State graphs with conditional edges for multi-agent anomaly diagnosis: Supervisor agent delegates to Diagnosis/Cost/Scheduling worker agents; compiled graphs with entry points
Agent Tools	LangChain @tool decorator	Tool definitions for sensor_query, manual_search, cost_calculator, schedule_maintenance; JSON Schema input schemas
HITL	LangGraph checkpointing (MemorySaver)	Human-in-the-loop approval gates on high-impact maintenance recommendations; conversation state persisted across interrupts

Continued on next page

Layer	Technology	Justification
MCP Integration	MCP SDK (Tools, Resources, Prompts)	Factory data exposed as MCP primitives: sensor readings as Resources, simulation as Tools, anomaly templates as Prompts. SSE transport for Claude Desktop client
Streaming	Async iterators via SSE	Token-by-token LLM response streaming to frontend via Server-Sent Events (SSE-Starlette)
API Layer		
Web Framework	FastAPI + Uvicorn (ASGI)	Async-native with automatic OpenAPI docs; Uvicorn for production ASGI serving
Streaming	SSE-Starlette	Server-Sent Events for real-time LLM token streaming and simulation progress updates
Validation	Pydantic v2 (BaseModel, Field, validators)	Request/response schema enforcement with field_validator, model_validator; Generic[T] for paginated responses
Type System	Enums, dataclasses, type hints	Literal types for anomaly urgency levels; Optional/Annotated for sensor readings; TypedDict for structured configs
Background Jobs	FastAPI BackgroundTasks	Async embedding refresh, report generation, and cache warming without blocking request handlers
Presentation Layer		
User Frontend	React + Recharts	Digital Twin UI: factory floor designer, simulation engine, scenario dashboards, AI advisor chat
Admin Dashboard	Streamlit	Operational dashboard with page config, sidebar filters, metrics cards, tabs, expanders; cache_data with TTL for sensor summaries
Visualization	Plotly Express	Interactive scatter plots (sensor clusters), size/color encoding for anomaly severity, reference lines for thresholds
Cross-Cutting Concerns		
Containerization	Docker + Docker Compose	Multi-service orchestration: FastAPI, Redis, Streamlit, Airflow; named volumes, depends_on, health checks, non-root user. Pinecone accessed as managed cloud service
Deployment	GCP Cloud Run	Serverless container hosting with auto-scaling; Dockerfile with HEALTHCHECK
Monitoring	Prometheus (Counter, Histogram, Gauge)	Custom metrics with labels: llm_request_duration, pipeline_rows_processed, cache_hit_ratio; time.perf_counter() for timing
Logging	structlog	Structured key-value logging with context variables and per-module loggers; JSON output for log aggregation

Continued on next page

Layer	Technology	Justification
Testing	pytest + Hypothesis + pytest-asyncio	Unit tests with fixtures (conftest.py); property-based testing (boundary, monotonicity, determinism); AsyncMock for LLM calls; coverage $\geq 80\%$
CI/CD	GitHub Actions	Automated test suite, ruff linting, mypy type checking, Docker build, deployment gates
Design Patterns	Repository/service, cache-aside, state machine	Repository layer for Snowflake access; service layer for business logic; cache-aside for Redis; state machine for simulation lifecycle; soft delete for scenarios
Config	.env, argparse, requirements.txt	Environment variable management; CLI scripts for data ingestion; pinned dependency versions

5.3 Architecture

Figure 1 presents the system architecture. The platform follows a six-layer design with clear separation of concerns: data sources, ingestion, processing, intelligence, API, and presentation.

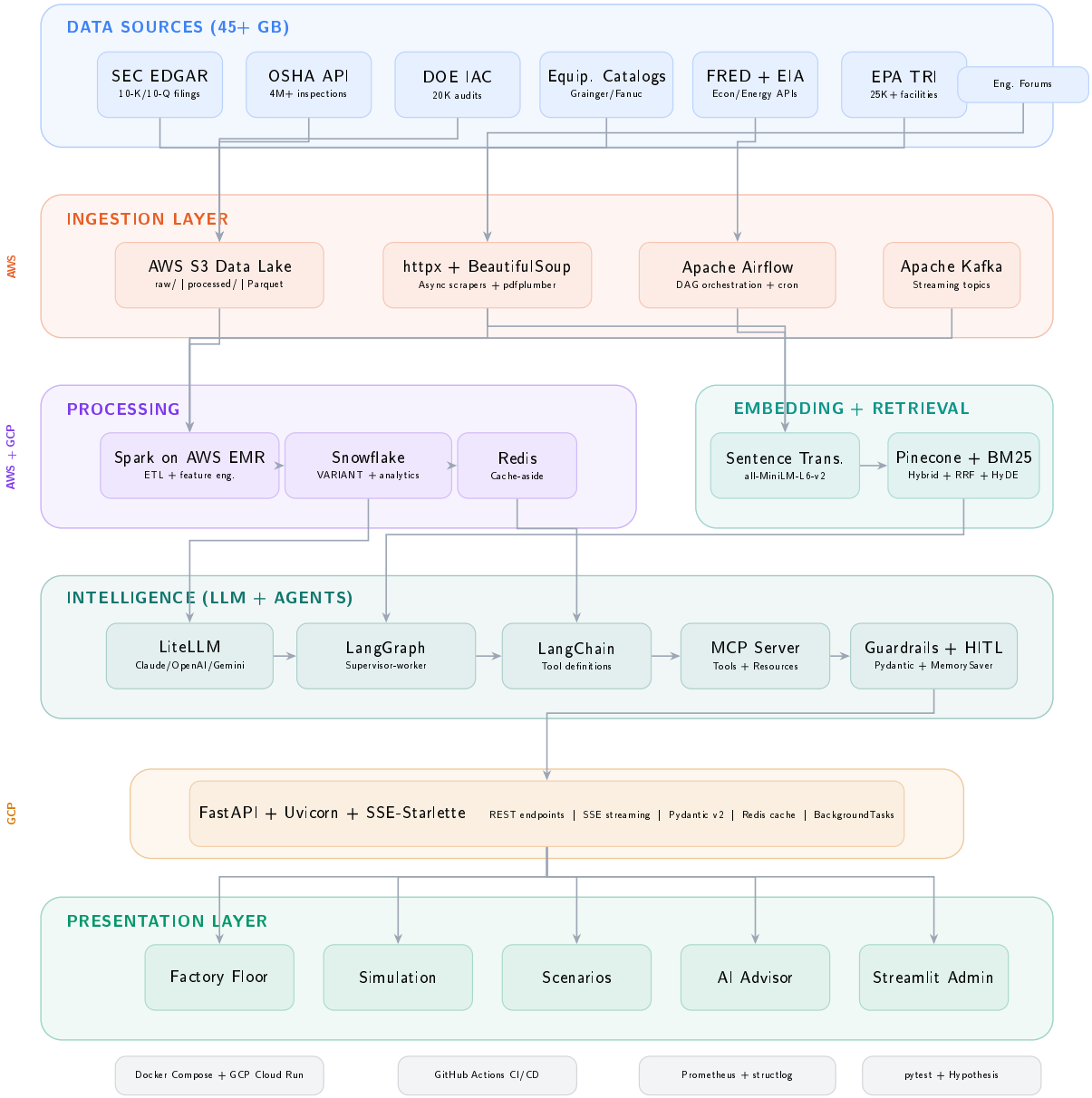

Figure 1: System Architecture Diagram — Seven-layer cloud-native Digital Twin platform with 45+ GB multi-source data pipeline, LangGraph agentic intelligence, and dual-frontend presentation

5.3.1 Data Flow Diagram

The data flows through the system in four primary paths:

1. **Batch Sensor Pipeline:** Raw datasets → S3 landing zone (`raw/`) → Airflow triggers Spark job → Schema normalization + feature engineering → S3 processed zone (`processed/`) in Parquet → BigQuery sync via scheduled transfer

2. **Streaming Sensor Pipeline:** IoT generator → Kafka topic → Spark Structured Streaming → Windowed aggregates (5min tumbling) → S3 micro-batch writes → Anomaly threshold check

→ LLM explanation trigger

3. **RAG Document Pipeline:** Equipment manuals (PDF) → PyMuPDF parser → Unstructured chunker (512 tokens, 64 overlap) → Metadata enrichment (equipment_type, section_type) → Voyage AI embedding → Pinecone batch upsert
4. **User Query Pipeline:** User input (chat or dashboard) → FastAPI → Context Builder (fetches recent sensor data from BigQuery + retrieves top-5 Pinecone chunks by equipment type) → Prompt assembly → Claude API call → Guardrail validation → HITL check (if urgency = "high") → Response to React UI

5.4 Data Processing & Transformation

5.4.1 Batch Processing (Spark on EMR)

- **Schema Normalization:** All sensor datasets transformed to a unified schema: `(timestamp, machine_id, sensor_type, value, unit, line_id, station_id)`. Spark UDFs handle type coercion, timezone normalization (to UTC), and missing value imputation (forward-fill for sensor readings, zero-fill for categorical).
- **Time-Series Feature Engineering:** Window functions compute rolling aggregates at three granularities (5min, 15min, 1hr): mean, standard deviation, min, max, rate of change ($\Delta v/\Delta t$), and coefficient of variation. For vibration data, FFT-based frequency features extract dominant frequencies and spectral energy.
- **Anomaly Scoring:** Three complementary anomaly detection methods computed per sensor per machine: (1) Z-score normalization against the machine's historical baseline, (2) Isolation Forest scores for multivariate anomaly detection, and (3) CUSUM change-point detection for identifying gradual drift.
- **Partitioning Strategy:** Processed data stored as Parquet files partitioned by `date/line_id/machine_id` for efficient pruning in BigQuery external table queries.
- **Data Quality:** Great Expectations validation suite integrated into Airflow DAGs: schema validation (column types, required fields), null rate checks (<5% per column), and statistical distribution drift detection (KL-divergence against the prior week's baseline).

5.4.2 Streaming Processing

- Python IoT generator produces JSON messages at 1 Hz per simulated machine via Kafka topic (`factory.sensors.raw`)
- Spark Structured Streaming consumes from Kafka with 5-minute tumbling windows
- Windowed aggregates (mean, max, anomaly score) written as micro-batches to S3 and synced to a BigQuery streaming buffer table
- When anomaly score exceeds threshold (configurable, default $Z > 3.0$), an event is published to a Kafka alert topic that triggers the LLM explanation pipeline

5.4.3 Embedding Generation Pipeline

- Equipment manuals parsed using `PyMuPDF` with layout-aware extraction preserving tables and diagrams as text descriptions

- `Unstructured` library handles section boundary detection and metadata extraction (headers, bullet lists, technical specifications)
- Recursive text splitting: 512-token chunks with 64-token overlap to preserve cross-paragraph context
- Each chunk enriched with metadata: `equipment_type`, `manufacturer`, `section_type` (troubleshooting, specifications, preventive maintenance, safety), `page_number`
- Batch embedding via Voyage AI API (100 chunks per batch), indexed into Pinecone with metadata filters
- Reindexing triggered by Airflow DAG on manual update detection (file hash comparison)

5.5 LLM Integration Strategy

The LLM layer is the platform's core intelligence engine, architecturally integrated across four workflows via a multi-provider, multi-agent system. This is not a simple API wrapper—it combines **LiteLLM** for provider routing, **LangGraph** for agentic state machines, **LangChain** for tool definitions, and **MCP** for external client integration.

5.5.1 Multi-Provider LLM Routing (LiteLLM)

All LLM calls route through **LiteLLM**, which provides:

- **Provider routing:** Claude Sonnet (primary) for complex RAG analysis, OpenAI GPT-4o (via `ChatOpenAI`) for NL-to-SQL translation, Claude Haiku for anomaly severity classification
- **Automatic fallbacks:** If Claude API is rate-limited or unavailable, requests fall back to OpenAI GPT-4o; if OpenAI is also degraded, Google Gemini serves as the tertiary fallback—ensuring zero downtime for the intelligence layer
- **Cost tracking:** Per-request token counting and budget management with configurable monthly limits
- **Per-task configuration:** Temperature=0.1 for SQL generation (deterministic), temperature=0.7 for scenario analysis (creative), max_tokens tuned per use case

5.5.2 Agentic Architecture (LangGraph)

The anomaly diagnosis pipeline uses a **LangGraph state graph** with a supervisor-worker pattern:

Listing 1: LangGraph multi-agent anomaly diagnosis

```python
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import MemorySaver

# State definition
class DiagnosisState(TypedDict):
    anomaly: AnomalyAlert
    sensor_context: dict
    manual_chunks: list[str]
    diagnosis: str | None
    cost_estimate: Decimal | None
    schedule: dict | None
```

```
    human_approved: bool

# Build graph
graph = StateGraph(DiagnosisState)
graph.add_node("supervisor", supervisor_agent)
graph.add_node("diagnosis", diagnosis_worker)     # RAG + Claude
graph.add_node("cost_analysis", cost_worker)      # Decimal math
graph.add_node("scheduling", schedule_worker)     # Maintenance slot
graph.add_node("human_review", hitl_gate)         # Approval gate

# Conditional routing
graph.add_conditional_edges("supervisor", route_by_urgency,
    {"critical": "diagnosis", "routine": "scheduling"})
graph.add_edge("diagnosis", "cost_analysis")
graph.add_conditional_edges("cost_analysis", check_cost_threshold,
    {"high": "human_review", "low": "scheduling"})
graph.add_edge("human_review", "scheduling")

app = graph.compile(checkpointer=MemorySaver())
```

The Supervisor agent receives anomaly alerts, classifies urgency, and delegates to specialist workers. **LangGraph conditional edges** route high-cost recommendations ($10K+ via Python Decimal) through a HITL approval gate. **MemorySaver checkpointing** persists state so the conversation survives interruption.

5.5.3 Tool Definitions (LangChain)

Each worker agent uses **LangChain @tool** decorated functions with typed schemas:

Listing 2: LangChain tool definitions

```
from langchain_core.tools import tool

@tool
def query_sensor_data(machine_id: str, hours: int = 24) -> dict:
    """Fetch recent sensor readings from Snowflake."""
    # Snowflake Python Connector query
    ...

@tool
def search_equipment_manual(query: str, equipment_type: str) -> list:
    """Hybrid search: Pinecone dense + BM25 sparse + RRF."""
    dense = pinecone_index.query(query, filter={"type": equipment_type})
    sparse = bm25_index.get_scores(tokenize(query))
    return reciprocal_rank_fusion(dense, sparse)

@tool
def calculate_maintenance_cost(parts: list, labor_hrs: float) -> str:
    """Precise cost with Python Decimal (ROUND_HALF_UP)."""
    ...
```

5.5.4 MCP Server Integration

The platform exposes factory data as **Model Context Protocol (MCP)** primitives, enabling Claude Desktop to interact with the Digital Twin directly:

- **MCP Tools:** `run_simulation(config)`, `query_sensors(machine_id, timerange)`, `compare_scenarios(id` callable from Claude Desktop via SSE transport
- **MCP Resources:** Live sensor readings, factory configuration, and scenario results exposed as subscribable resources with `inputSchema` (JSON Schema)
- **MCP Prompts:** Pre-built prompt templates for anomaly diagnosis, shift optimization, and maintenance scheduling
- **Transport:** SSE (Server-Sent Events) transport layer for real-time streaming; Claude Desktop configured as MCP client

5.5.5 Use Case Summary

Use Case	LLM	Framework	How It Works
Anomaly Explanation	Claude Sonnet	LangGraph + RAG	Supervisor routes to Diagnosis worker; Pinecone hybrid retrieval (dense + BM25 + RRF + HyDE); Pydantic structured output
NL-to-SQL Queries	GPT-4o / Gemini fallback	LangChain	User question → SQL generation → Snowflake execution → natural language summary
Scenario Analysis	Claude Sonnet	LangChain @tool	Simulation results + Claude analysis; Python Decimal for cost precision; configurable recommendations
Weekly Reports	Claude Haiku + Sonnet	Airflow + LiteLLM	Haiku classifies alert severity; Sonnet generates narrative; Airflow DAG schedules weekly
Desktop Integration	Claude Desktop	MCP SDK	Factory data as MCP Tools/Resources; engineers query the twin from Claude Desktop directly

Table 5: LLM use cases mapped to providers and frameworks

5.6 Guardrails & Human-in-the-Loop (HITL)

Layer	Mechanism	Description
Input	Pydantic validation	All API inputs validated for types, ranges (sensor values within physical bounds), and required fields
Input	Prompt injection screening	Input text screened for adversarial patterns before Claude API calls
Input	Rate limiting	Per-session token budget (~100K tokens/session) to prevent cost overruns
Output	JSON schema enforcement	All LLM structured outputs validated against Pydantic models; malformed responses trigger retry
Output	Hallucination detection	LLM-cited sensor values cross-referenced against actual BigQuery values; discrepancies flagged
Output	Confidence scoring	Claude provides confidence (high/medium/low) for each recommendation; "low" triggers human review
Output	Toxicity filtering	Free-form text outputs screened for inappropriate content
HITL	Urgency-based approval	Recommendations with urgency = "critical" or "high" require human approval before escalation
HITL	Cost threshold	Any recommendation with estimated cost impact > $10,000 triggers mandatory review
HITL	Report staging	Automated weekly reports published as "Draft" for human review before distribution
HITL	Config confirmation	AI-suggested factory configuration changes require explicit user click to apply

Table 6: Guardrails and HITL mechanisms by layer

5.7 Evaluations & Testing

5.7.1 LLM Evaluation Framework

- **Golden Set:** 50 curated anomaly scenarios with expert-labeled root causes, recommended actions, and urgency levels. Used as ground truth for RAG accuracy evaluation.
- **Rubric-Based Grading:** Each LLM response scored on four dimensions: relevance (0–5), accuracy (0–5), actionability (0–5), and hallucination (binary). Automated via a separate Claude instance acting as judge.
- **RAG Retrieval Metrics:** Precision@5, Recall@5, Mean Reciprocal Rank (MRR) for vector search quality. Evaluated against human-annotated "ideal chunk" labels.
- **NL-to-SQL Accuracy:** 30 test queries with expected SQL output. Measured by exact match and execution result equivalence.
- **Regression Testing:** Golden set re-evaluated on every prompt template change via GitHub Actions.

5.7.2 Software Testing

- **Unit Tests:** pytest with fixtures (conftest.py) for Spark transformations, API endpoints, LLM prompt builders, Pydantic validators, and simulation engine. AsyncMock for mocking LLM API calls. Target: >80% coverage.

- **Property-Based Tests:** Hypothesis framework for testing invariants—boundary conditions on sensor value normalization, monotonicity of degradation scores, determinism of simulation engine given fixed seeds, and cost calculation precision (Decimal ROUND_HALF_UP).
- **Integration Tests:** End-to-end pipeline (S3 → Spark → Snowflake) and API→LangGraph→LLM→Response chain tests with mocked LLM responses via pytest-asyncio.
- **Load Tests:** Locust-based stress testing of FastAPI under concurrent simulation and chat requests.
- **CI Pipeline:** GitHub Actions with automated test suite, ruff linting, mypy type checking, and deployment gates.

5.7.3 Key Metrics

Metric	Target	Measurement
RAG Answer Accuracy	>80% on golden set	Expert eval + LLM-as-Judge
Anomaly Explanation Relevance	>4.0/5.0 average	Rubric-based scoring
NL-to-SQL Accuracy	>75% exact match	Golden query set
API Response Latency (P95)	<3 seconds	Prometheus
Spark Pipeline Runtime (Bosch)	<15 minutes	Airflow task duration
Hallucination Rate	<5%	Fact-checking against BigQuery
Test Coverage	>80%	pytest-cov
System Uptime	>99.5%	GCP Cloud Monitoring

Table 7: Key performance metrics and targets

5.8 Proof of Concept (POC)

The following POC artifacts are planned for Week 1 and will be committed to the project repository at https://github.com/BigDataIA-Spring26-Team-1/FinalProjectProposal:

1. **React Digital Twin Frontend:** 4-tab prototype application—Factory Floor (visual production line designer), Simulation (time-based engine with Recharts dashboards), Scenarios (save/-compare configurations), AI Advisor (LLM-powered chat). Initial scaffold with mock data to validate the UI architecture before backend integration.
2. **SEC EDGAR Scraping Pipeline:** Pipeline using `sec-edgar-downloader` + `httpx` to fetch 10-K filings from 10 manufacturing companies. `pdfplumber` text extraction; regex-based section parser to isolate capex, depreciation, and maintenance expense disclosures. Initial EDA in Jupyter.
3. **OSHA API Ingestion Prototype:** Paginated REST API client pulling inspection records filtered by NAICS manufacturing codes (31–33). SHA-256 dedup; initial batch loaded into Snowflake staging table with VARIANT column. Schema normalization scripts.
4. **Pinecone + BM25 Hybrid RAG Prototype:** Small-scale test with ~50 equipment manual pages scraped via `httpx` + `BeautifulSoup`, chunked, embedded via `all-MiniLM-L6-v2`, and indexed in Pinecone. BM25 sparse index via `rank_bm25`. Validate Reciprocal Rank Fusion retrieval quality before scaling.

5. **Airflow DAG Skeleton:** Draft DAG with task definitions for OSHA API ingestion, SEC EDGAR scraping trigger, Spark ETL submission via EMR, Pinecone embedding refresh, Snowflake sync, and weekly LLM report generation. Retry policies with exponential backoff.
6. **Docker Compose Environment:** Multi-service `docker-compose.yml` orchestrating FastAPI (Uvicorn), Redis, and Streamlit admin dashboard with `depends_on`, named volumes, health checks, and environment variable passthrough. Pinecone accessed via API key.

6 Project Plan & Timeline

6.1 Milestones

ID	Phase	Timeline	Deliverables
M1	Data Ingestion + Processing	Week 1 (Days 1–4)	S3 data lake setup; SEC EDGAR + OSHA + DOE scraping pipelines; Spark ETL on EMR; Snowflake schema; Airflow DAGs; Great Expectations validation
M2	Embedding + RAG Pipeline	Week 1 (Days 3–7)	Manual/forum scraping (httpx + BeautifulSoup); chunking + embedding (Sentence Transformers); Pinecone + BM25 hybrid index; HyDE + RRF retrieval; FRED/EIA API ingestion
M3	LLM Agentic Integration	Week 2 (Days 1–4)	LangGraph supervisor-worker graph; LiteLLM multi-provider routing (Claude + OpenAI + Gemini); LangChain @tool definitions; MCP server (Tools, Resources, Prompts); guardrails + HITL approval gates
M4	Backend + Frontend	Week 2 (Days 3–7)	FastAPI + Uvicorn + SSE-Starlette endpoints; Redis cache-aside; React Digital Twin UI integration; Streamlit admin dashboard; Docker Compose orchestration
M5	Cloud Deploy + Testing	Week 3 (Days 1–4)	GCP Cloud Run deployment; AWS EMR production config; CI/CD via GitHub Actions; pytest + Hypothesis test suite; LLM golden set evaluation; Prometheus + structlog monitoring
M6	Polish + Documentation	Week 3 (Days 4–7)	CodeLabs document; video presentation (10 min, all members); README; final optimizations; EPA TRI cross-joins; cost report finalization

Table 8: Project milestones and timeline

6.2 Timeline

A 3-week sprint timeline is maintained in **GitHub Projects** with Kanban boards for daily tracking. All three team members work in parallel across milestones, with daily standups and PR-based code

review. Board accessible at: https://github.com/BigDataIA-Spring26-Team-1/FinalProjectProposal/projects.

7 Team Roles & Responsibilities

Role	Member	Responsibilities
ETL & Data Lead	Ayush Patil	Spark pipeline development; Airflow DAGs; OSHA/EPA/DOE data ingestion (API pagination, SHA-256 dedup); Snowflake schema design (VARIANT, compound constraints); Kafka streaming pipeline; Great Expectations data quality suite; S3 data lake management
LLM & Cloud Architect	Raghavendra Prasath Sridhar	LangGraph agentic workflows (supervisor-worker, conditional edges); LiteLLM multi-provider routing (Claude + OpenAI + Gemini fallback chain); prompt engineering; Pinecone + BM25 hybrid RAG pipeline; MCP server (Tools, Resources, Prompts); guardrails and HITL; AWS EMR + GCP Cloud Run deployment
Frontend & QA Lead	Piyush Kunjilwar	React Digital Twin UI; Streamlit admin dashboard; FastAPI backend (Uvicorn, SSE-Starlette streaming); SEC EDGAR scraping + equipment catalog crawling (httpx, BeautifulSoup); Docker Compose orchestration; pytest + Hypothesis testing; CI/CD pipeline; CodeLabs writeup; video presentation

Table 9: Team roles and primary responsibilities

All members contribute to code review, testing, and documentation. Roles define primary ownership, not exclusive responsibility.

8 Risks & Mitigation

8.1 Risk Assessment Matrix

Risk	Severity	Likelihood	Mitigation Strategy
LLM hallucinations in maintenance recommendations	High	Medium	Confidence scoring; HITL approval for high-impact; fact-checking against DB; golden set regression
High Claude API costs exceeding budget	Medium	High	Prompt caching (90% savings on cache hits); response caching (Redis); token budgets per session; Haiku for simple tasks
Spark failures on large Bosch dataset	Medium	Medium	Airflow retry with exponential backoff; Spark checkpointing; incremental processing fallback
Poor RAG retrieval quality	Medium	Medium	Hybrid retrieval (vector + BM25); metadata filtering; chunk overlap tuning; cross-encoder re-ranking
Schema drift across data sources	Low	High	Great Expectations validation; schema version tracking; automated alerts on violations
GCP/AWS service outages	Low	Low	Multi-region data replication; health checks; auto-scaling; Terraform for reproducible infrastructure

Table 10: Risk assessment and mitigation strategies

9 Expected Outcomes & Metrics

9.1 KPIs

KPI	Baseline (Manual)	Target (Platform)	Improvement
Anomaly Root Cause Time	4–48 hours	<5 minutes	98% ↓
Sensor Data Processing	Manual exports, hours	<15 min (50+ GB)	Automated
Simulation Turnaround	Days (physical test)	<30 seconds	Real-time
Maintenance Report Gen.	2–4 hours (manual)	<2 minutes (LLM)	95% ↓
Data Source Integration	1 at a time, siloed	6 sources unified	Complete
Natural Language Querying	N/A (SQL required)	Plain English	New capability

Table 11: Key performance indicators with baseline vs. target comparison

9.2 Expected Benefits

- **Technical:** Demonstrates end-to-end big data pipeline architecture (ingestion → distributed processing → LLM intelligence → interactive presentation) at production scale with cloud-native tooling and modern MLOps practices.

- **Business:** Reduces unplanned downtime through predictive analytics; cuts maintenance response time from hours to minutes; enables data-driven capital expenditure decisions through what-if simulation; preserves institutional knowledge in a searchable AI system.
- **Academic:** Integrates distributed computing (Spark), modern LLM patterns (RAG, structured outputs, guardrails, HITL), multi-cloud architecture (AWS + GCP), and user-facing application development in a cohesive, well-tested platform.

10 Token & Cost Report

10.1 Token Consumption Tracking

All Claude API calls are logged with: timestamp, endpoint, model (Sonnet/Haiku), `input_tokens`, `output_tokens`, `cache_creation_tokens`, `cache_read_tokens`, and calculated cost. A Grafana dashboard provides real-time visibility into cumulative usage, cost per feature, and daily trends.

10.2 Cost Drivers

1. **Primary:** RAG anomaly explanation calls—large context with sensor data summary + 5 retrieved manual chunks + system prompt (~20K input tokens per call)
2. **Secondary:** Natural language query translation (~5K tokens per call)
3. **Tertiary:** Weekly automated report generation (~30K tokens per report)

10.3 Optimization Strategy

- **Prompt Caching:** System prompt + factory context (rarely changes) cached via Claude's prompt caching API, reducing repeat input costs by up to 90%
- **Response Caching:** Redis cache for identical queries within a 15-minute window (common in dashboard refresh scenarios)
- **Model Tiering:** Claude Haiku for simple classification (anomaly severity routing, query type detection); Sonnet for complex analysis and RAG
- **Embedding Batching:** Chunks grouped in batches of 100 to minimize API call overhead
- **Prompt Compression:** Sensor data sent as statistical summaries (mean, std, trend direction) rather than raw readings, reducing token count by ~70%

10.4 Estimated Budget

Component	Est. Monthly Volume	Est. Cost
Claude Sonnet (RAG + Analysis)	~2M input / 500K output tokens	$12–18
OpenAI GPT-4o (NL-to-SQL + fallback)	~800K input / 200K output tokens	$5–8
Google Gemini (tertiary fallback)	~200K input / 50K output tokens	$1–2
Claude Haiku (Classification)	~500K input / 100K output tokens	$1–2
Voyage AI Embeddings	~1M tokens	$1–2
Pinecone Vector Store	~100K vectors, 10K queries/mo	$0 (free tier)
AWS EMR (Spark)	~20 compute hours	$15–25
GCP Cloud Run	~100K requests/month	$5–10
AWS S3 Storage	~100 GB stored	$2–3
GCP BigQuery	~500 GB scanned/month	$2–4
Total Estimated		**$45–72/month**

Table 12: Estimated monthly infrastructure and API costs

11 Conclusion

This project delivers a production-grade Digital Twin platform that addresses a critical gap in modern manufacturing: the absence of accessible, intelligent, data-driven factory simulation and analytics tools. By combining big data engineering (Apache Spark on EMR, Airflow orchestration, S3 data lake, Snowflake warehouse), cutting-edge LLM capabilities (multi-agent LangGraph workflows, RAG with hybrid retrieval via Pinecone + BM25 + RRF, LiteLLM multi-provider routing, MCP server integration), and multi-cloud deployment (AWS + GCP with Docker Compose orchestration), the platform transforms 50+ GB of raw industrial sensor data into actionable manufacturing intelligence.

The LLM integration is not superficial—it spans a full agentic architecture: LangGraph state graphs with supervisor-worker patterns and HITL approval gates, LangChain tool definitions for sensor queries and manual search, LiteLLM for intelligent provider routing with cost tracking, and an MCP server that lets Claude Desktop interact with the factory twin directly. Each workflow includes guardrails (Pydantic v2 validation, hallucination detection, confidence scoring) and structured logging via structlog.

The React-based Digital Twin frontend provides an intuitive interface where factory managers can visually design production lines, run time-based simulations calibrated with real sensor data, compare scenarios with multi-dimensional dashboards, and consult an AI-powered manufacturing advisor—all without touching the physical factory floor. The platform demonstrates the full spectrum of big data and intelligent analytics, from raw data ingestion at scale to intelligent, user-facing decision support.

12 References

1. SEC EDGAR Full-Text Search and EDGAR Filing API. https://efts.sec.gov/LATEST/search-index?q=
2. OSHA Enforcement Data — Inspection, Accident, and Violation Records. https://enforcedata.dol.gov/views/data_summary.php
3. U.S. DOE Industrial Assessment Center (IAC) Database. https://iac.university/
4. FRED — Federal Reserve Economic Data API. https://fred.stlouisfed.org/docs/api/fred/
5. U.S. Energy Information Administration (EIA) Open Data API. https://www.eia.gov/opendata/
6. EPA Toxics Release Inventory (TRI) Program. https://www.epa.gov/toxics-release-inventory-tri-prog
7. Grainger Industrial Supply — Product Catalog. https://www.grainger.com/
8. Anthropic Claude API Documentation. https://docs.anthropic.com/
9. LangGraph — Multi-Agent Orchestration Framework. https://langchain-ai.github.io/langgraph/
10. LiteLLM — LLM Provider Routing and Cost Tracking. https://docs.litellm.ai/
11. OpenAI API Documentation. https://platform.openai.com/docs/
12. Google Gemini API Documentation. https://ai.google.dev/docs
13. Model Context Protocol (MCP) Specification. https://modelcontextprotocol.io/
14. Pinecone — Managed Vector Database. https://docs.pinecone.io/
15. Snowflake Python Connector Documentation. https://docs.snowflake.com/en/developer-guide/python-connector/
16. Apache Spark Documentation. https://spark.apache.org/docs/latest/
17. Apache Airflow Documentation. https://airflow.apache.org/docs/
18. Sentence Transformers (all-MiniLM-L6-v2). https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
19. Grieves, M. "Digital Twin: Manufacturing Excellence through Virtual Factory Replication," White Paper, 2014.
20. McKinsey Global Institute. "The Internet of Things: Mapping the Value Beyond the Hype," 2015.
21. Tao, F. et al. "Digital twin-driven product design, manufacturing and service with big data." *Int. J. Adv. Manuf. Technol.*, 94(9), 3563–3576, 2018.
22. Project Repository: https://github.com/BigDataIA-Spring26-Team-1/FinalProjectProposal

Appendix

A. System Architecture Diagram (Mermaid)

The following Mermaid diagram can be rendered in the project README:

Listing 3: Mermaid architecture diagram for README.md

```
graph TD
    subgraph DS["Data Sources (45+ GB)"]
        A1[SEC EDGAR 10-K Filings<br/>50+ manufacturers]
        A2[OSHA Inspections<br/>4M+ records]
        A3[DOE IAC Database<br/>20K audits]
        A4[Equipment Catalogs<br/>Grainger/Fanuc/ABB]
        A5[FRED + EIA APIs<br/>Economic + Energy]
        A6[EPA TRI<br/>25K+ facilities]
        A7[Engineering Forums<br/>Stack Exchange/Reddit]
    end

    subgraph IL["Ingestion Layer"]
        B1[AWS S3 Data Lake<br/>raw/ | processed/]
        B2[httpx + BeautifulSoup<br/>Async scrapers]
        B3[Apache Airflow<br/>DAG orchestration]
    end

    subgraph PL["Processing Layer"]
        C1[Spark on AWS EMR<br/>ETL + Feature Eng.]
        C2[Snowflake<br/>VARIANT + Analytical]
        C3[Redis<br/>Cache-aside + TTL]
    end

    subgraph EP["Embedding + Retrieval"]
        D1[Sentence Transformers<br/>all-MiniLM-L6-v2]
        D2[Pinecone + BM25<br/>Hybrid + RRF + HyDE]
    end

    subgraph INT["Intelligence Layer"]
        E1[LiteLLM<br/>Multi-provider routing]
        E2[LangGraph<br/>Supervisor-worker agents]
        E3[MCP Server<br/>Tools + Resources]
        E4[Guardrails + HITL<br/>Pydantic + MemorySaver]
    end

    subgraph API["API Layer"]
        F1[FastAPI + Uvicorn<br/>SSE-Starlette + Pydantic v2]
    end

    subgraph UI["Presentation Layer"]
        G1[React Digital Twin UI]
        G2[Streamlit Admin Dashboard]
    end

    A1 & A2 & A3 & A6 --> B1
```

```
    A4 & A7 --> B2
    A5 --> B3
    B1 & B2 --> C1
    B3 --> C1
    C1 --> C2
    C2 --> C3
    B2 --> D1
    D1 --> D2
    C2 --> E1
    D2 --> E2
    E1 --> E2
    E2 --> E4
    E3 --> F1
    E4 --> F1
    F1 --> G1 & G2
```

B. Pydantic Response Schema

Listing 4: Anomaly explanation Pydantic schema

```python
from pydantic import BaseModel, Field
from typing import Literal

class AnomalyExplanation(BaseModel):
    root_cause: str = Field(
        ..., description="Primary root cause hypothesis"
    )
    confidence: Literal["high", "medium", "low"] = Field(
        ..., description="Confidence in the diagnosis"
    )
    evidence: list[str] = Field(
        ..., description="Supporting evidence from sensors/manuals"
    )
    recommended_action: str = Field(
        ..., description="Specific maintenance action to take"
    )
    urgency: Literal["critical", "high", "medium", "low"] = Field(
        ..., description="How quickly action is needed"
    )
    estimated_cost: float | None = Field(
        None, description="Estimated cost of recommended action"
    )
    requires_hitl: bool = Field(
        ..., description="Whether human approval is required"
    )
    manual_references: list[str] = Field(
        default_factory=list,
        description="Relevant manual section references"
    )
```

C. Sample Airflow DAG Structure

Listing 5: Simplified Airflow DAG for the main ETL pipeline

```python
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.amazon.aws.operators.emr import (
    EmrAddStepsOperator
)
from datetime import datetime, timedelta

default_args = {
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
}

with DAG(
    "digital_twin_etl",
    default_args=default_args,
    schedule_interval="@daily",
    start_date=datetime(2026, 1, 1),
    catchup=False,
) as dag:

    ingest = PythonOperator(
        task_id="ingest_to_s3",
        python_callable=ingest_sensor_data,
    )
    spark_etl = EmrAddStepsOperator(
        task_id="spark_feature_engineering",
        job_flow_id=EMR_CLUSTER_ID,
        steps=SPARK_STEPS,
    )
    quality_check = PythonOperator(
        task_id="great_expectations_validation",
        python_callable=run_data_quality_checks,
    )
    bq_sync = PythonOperator(
        task_id="sync_to_bigquery",
        python_callable=sync_parquet_to_bigquery,
    )
    embed_refresh = PythonOperator(
        task_id="refresh_embeddings",
        python_callable=update_pinecone_index,
    )

    ingest >> spark_etl >> quality_check >> bq_sync
    quality_check >> embed_refresh
```